

COMS / SE 4170 Software Testing Spring 2026

Chapter 2 – Model Driven Test Development

1

Correction from Syllabus

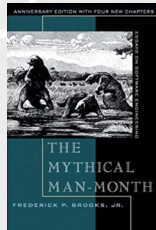
- There were **conflicting dates** in Canvas for Exam 2.
- Exam 2 is scheduled for **April 28th (Tuesday)**. Should be corrected in Canvas and lecture notes

2

Key Challenge

*"Software is inherently complicated and our ultimate goal, **completely correct software is unreachable**"*

Accidental versus essential complexity



3

Key Challenge

*"Software is inherently complicated and our ultimate goal, **completely correct software is unreachable**"*

If we can't reach this goal, we need to be able to **observe that our software is acceptable** with respect to **"reliability, safety, maintainability, security, efficiency"**

4

Limitations

- Testing can only **show the presence of faults**. It **cannot** determine the absence of faults.



5

Model-Driven Test Design

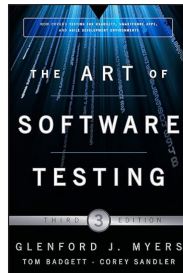
We can use higher levels of abstraction or **models** to reduce complexity and guide testing

- Helps with test case **design**
 - Use human knowledge and/or coverage
- Test case **generation**
- Test **sufficiency** (how much behavior/code have we covered)
- Test **prioritization** (which tests to run first)

6

The Art of Software Testing

- 1979 – Glenford Myers



7

Introduced the “Triangle Program”

- This has become a classic example
 - We will use some in this class

Now, before we start the book, we'd like you to take a short exam. We want you to write a set of test cases—specific sets of data—to test properly a relatively simple program. Create a set of test data for the program—data the program must handle correctly to be considered a successful program. Here's a description of the program:

The program reads three integer values from an input dialog. The three values represent the lengths of the sides of a triangle. The program displays a message that states whether the triangle is scalene, isosceles, or equilateral.

8

Introduced the “Triangle Program”

- This has become a classic example
 - We will use some in this class

Remember that a scalene triangle is one where no two sides are equal, whereas an isosceles triangle has two equal sides, and an equilateral triangle has three sides of equal length. Moreover, the angles opposite the

A Self-Assessment Test

equal sides in an isosceles triangle also are equal (it also follows that the sides opposite equal angles in a triangle are equal), and all angles in an equilateral triangle are equal.

9

Running Example Program

Triangle Program:

Input: 3 integer values

Output: invalid, equilateral, isosceles, or scalene

```
[mcohen@pyrite-n2 triangle]$ java triangle
4 4 4
EQUILATERAL
[mcohen@pyrite-n2 triangle]$ java triangle
4 5 6
SCALENE
[mcohen@pyrite-n2 triangle]$ java triangle
4 5 4
ISOSCELES
[mcohen@pyrite-n2 triangle]$ java triangle
1 1 0
Invalid
[mcohen@pyrite-n2 triangle]$
```

10

What Tests Should We Run?

- In Class discussion...

11

Running Example Program

- What makes a valid triangle (**specifications**)?
- What do we know about the code structure?
- Not always possible to have access to code

12

```

public static TriangleType triangle (int s1, int s2, int s3)
{
    // Reject non-positive sides
    if (s1 <= 0 || s2 <= 0 || s3 <= 0)
        return (Triangle.INVALID);

    // Check triangle inequality
    if (s1+s2 <= s3 || s2+s3 <= s1 || s1+s3 <= s2)
        return (Triangle.INVALID);

    // Identify equilateral triangles
    if ((s1 == s2) && (s2 == s3))
        return Triangle.EQUILATERAL;

    // Identify isosceles
    if ((s1 == s3) || (s2 == s3) || (s1 == s2))
        return Triangle.ISOSCELES;

    return (Triangle.SCALENE);
}

```

should be s1==s2

13

```

[mcohen@pyrite-n2 triangle]$ java triangle
5 5 6
SCALENE
[mcohen@pyrite-n2 triangle]$ █

```

14

Some Foundations

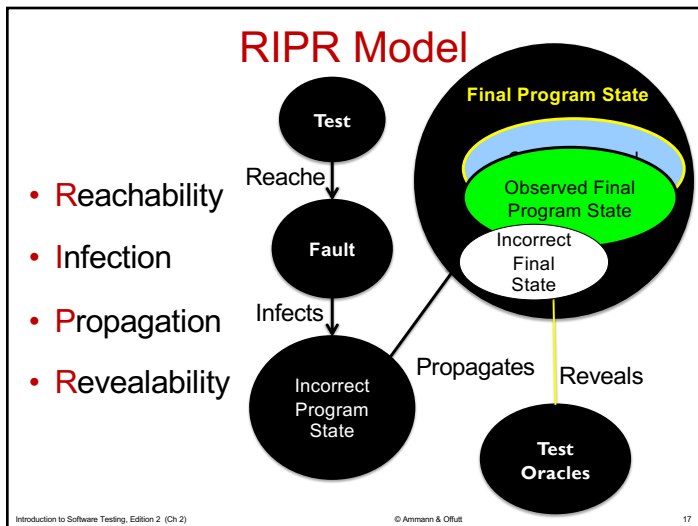
- **Testing**
 - Exercising software and observing/evaluating its execution
- **Test Failure**
 - Execution of a test that results in an observable incorrect result (failure)
- **Debugging**
 - The process of finding a fault given a failure

15

Central Idea

- Not all inputs trigger a fault into an observable failure
- Difficult to relate a failure to an associated fault

16



17

Tests Can Easily Miss Faults

```

def classify_triangle(a, b, c):
    # Sort the sides so that a <= b <= c
    if a > b:
        tmp = a
        a = b
        b = tmp
    if a > c:
        tmp = a
        a = c
        c = tmp
    if b > c:
        tmp = b
        b = c
        c = tmp
    if a + b <= c:
        return TriangleType.INVALID
    elif a == b and b == c:
        return TriangleType.EQUILATERAL
    elif a == b or b == c:
        return TriangleType.ISOSCELES
    else:
        return TriangleType.SCALENE
  
```

18

Tests Can Easily Miss Faults

```

def classify_triangle(a, b, c):
    # Sort the sides so that a <= b <= c
    if a > b:
        tmp = a
        a = tmp
        b = tmp
    if a > c:
        tmp = a
        a = c
        c = tmp
    if b > c:
        tmp = b
        b = c
        c = tmp
    if a + b <= c:
        return TriangleType.INVALID
    elif a == b and b == c:
        return TriangleType.EQUILATERAL
    elif a == b or b == c:
        return TriangleType.ISOSCELES
    else:
        return TriangleType.SCALENE
  
```

- Test Case 3, 4, 5 (scalene)
 - doesn't reach fault

19

Tests Can Easily Miss Faults

```

def classify_triangle(a, b, c):
    # Sort the sides so that a <= b <= c
    if a > b:
        tmp = a
        a = tmp
        b = tmp
    if a > c:
        tmp = a
        a = c
        c = tmp
    if b > c:
        tmp = b
        b = c
        c = tmp
    if a + b <= c:
        return TriangleType.INVALID
    elif a == b and b == c:
        return TriangleType.EQUILATERAL
    elif a == b or b == c:
        return TriangleType.ISOSCELES
    else:
        return TriangleType.SCALENE
  
```

- Test Case 3, 4, 5 (scalene)
 - doesn't reach fault
- Test Case 2, 1, -1 (invalid)
 - reaches fault and infects
 - Doesn't propagate (2, 2, -1) is still **INVALID**

20

Tests Can Easily Miss Faults

```
def classify_triangle(a, b, c):
    # Sort the sides so that a <= b <= c
    if a > b:
        tmp = b
        a = tmp
        b = tmp
        # fault should be a=b
        # 5, 5, 1
    if a > c:
        tmp = a
        a = c
        c = tmp
    if b > c:
        tmp = b
        b = c
        c = tmp
    if a + b <= c:
        return TriangleType.INVALID
    elif a == b and b == c:
        return TriangleType.EQUILATERAL
    elif a == b or b == c:
        return TriangleType.ISOSCELES
    else:
        return TriangleType.SCALENE
```

1. Test Case 3, 4, 5 (scalene)
 - doesn't reach fault
2. Test Case 2, 1, -1 (invalid)
 - reaches fault and infects
 - Doesn't propagate (2, 2, -1) is still INVALID
3. Test Case 5, 1, 1 (invalid)
 - reaches fault and infects
 - reveals (returns isosceles)

✗

✗

✓

21

Fault/Failure Model

- **Reachability**– test must execute the faulty code.
- **Infection** – execution must create an incorrect state of the program (error)
- **Propagation**- the error has to propagate to the output or final state of program
- **Revealability**- the tester has to observe the incorrect output/final state

22

Testing Activities

- **Test Engineer:**
 - Designing test inputs
 - Producing test values
 - Running test scripts
 - Analyzing results
 - Reporting results to developers and manager
- **Test Manager:**
 - Sets test policies and processes
 - Interacts with other managers on the project
 - Otherwise supports the engineers

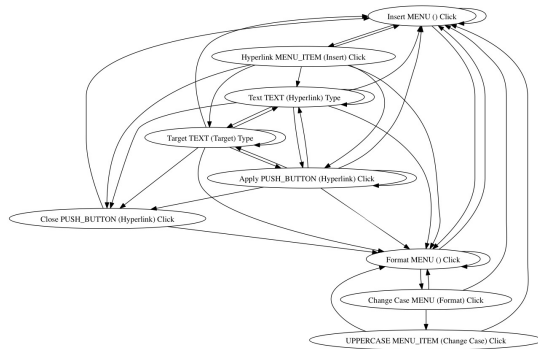
23

Coverage Criteria

- Helps to decide
 - Inputs
 - Stopping rules -- when have I tested enough?

24

Example Models Graphical Interface



25

Example Models Program Spec Partitioning

Parameters:

Pattern size:

- empty
- single character
- many character
- longer than any line in the file

Quoting:

- pattern is quoted
- pattern is not quoted
- pattern is improperly quoted

Embedded blanks:

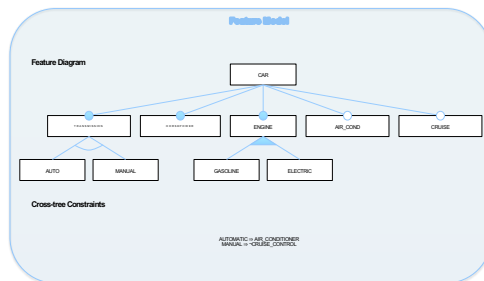
- No embedded blank
- One embedded blank
- Several embedded blanks

Embedded quotes:

- No embedded quotes
- One embedded quote
- Several embedded quotes

26

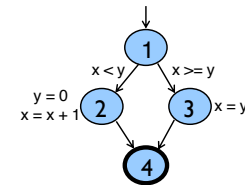
Example Models Program Features



27

Example Models Source Code

```
if (x < y)
{
  y = 0;
  x = x +
1;
}
else
{
  x = y;
}
```



Control flow graph

28

Using a CFG

Coverage Report			
Classes in this File	Line Coverage	Branch Coverage	Complexity
TriangleType	90% 9/10	81% 18/22	16

```

1 //Introduction to Software Testing
2 //Authors: Paul Ammann & Jeff Offutt
3 //modified by M.Cohen for class exercise
4 //Chapter 6;
5 // See TriangleTypeTest for Junit tests
6
7
8
9
10 0 public class TriangleType
11 {
12     /**
13      * @param s1, s2, s3: sides of the putative triangle
14      * @return enum describing type of triangle
15      */
16     public static Triangle triangle (int s1, int s2, int s3)
17     {
18         // Reject non-positive sides
19 11 if (s1 <= 0 || s2 <= 0 || s3 <= 0)
20 2     return (Triangle.INVALID);

```

29

Levels of Testing (top down)

- **Acceptance testing** – satisfies requirements/user needs. Must involve users with domain knowledge

30

Levels of Testing (top down)

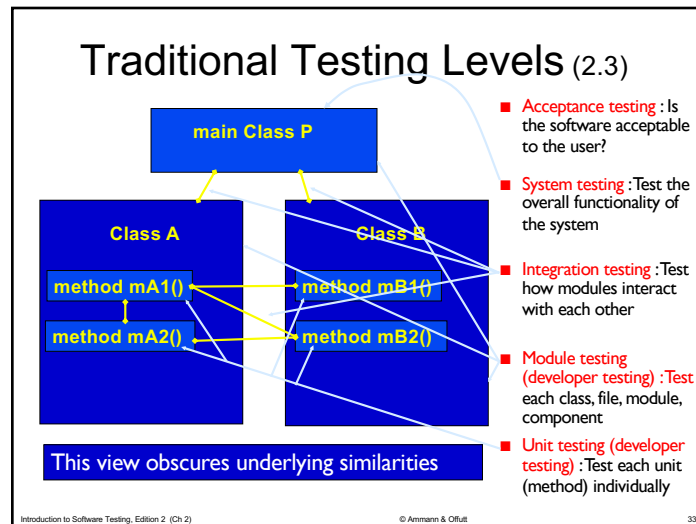
- **Acceptance testing** – satisfies requirements/user needs. Must involve users with domain knowledge
- **System testing** – evaluates overall system behavior. Assumes individual parts already tested. Usually tested by team independent from developers

31

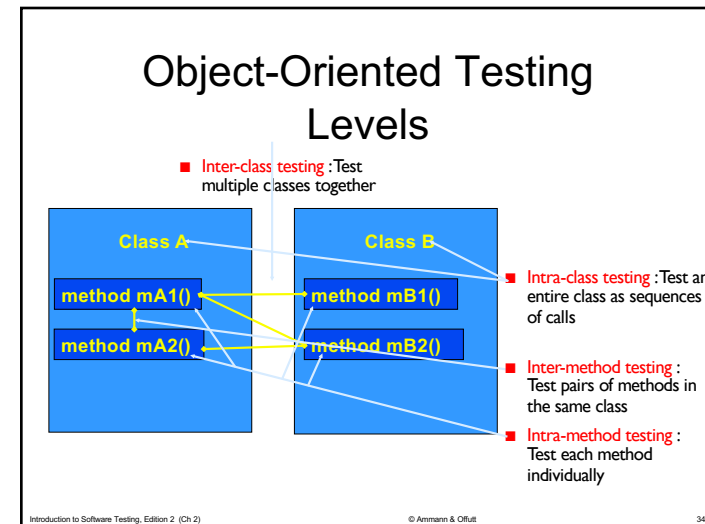
Developer testing levels

- **Integration testing** – evaluates subsystem design (puts modules together)
- **Module testing** – evaluates design of individual modules
- **Unit testing** – tests individual functions/units with respect to implementation

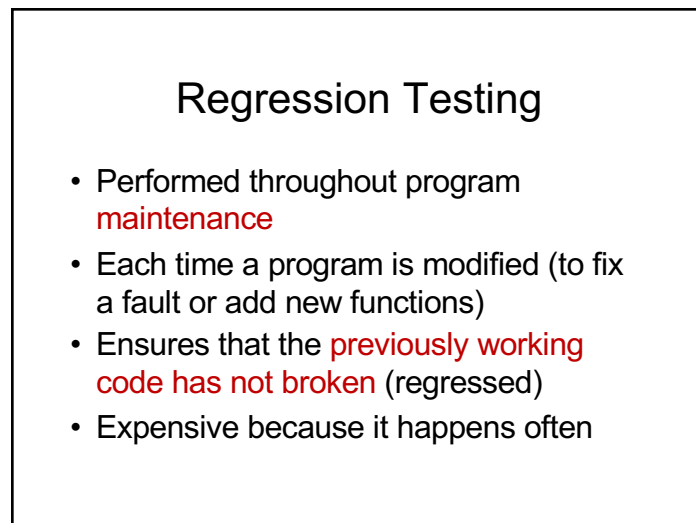
32



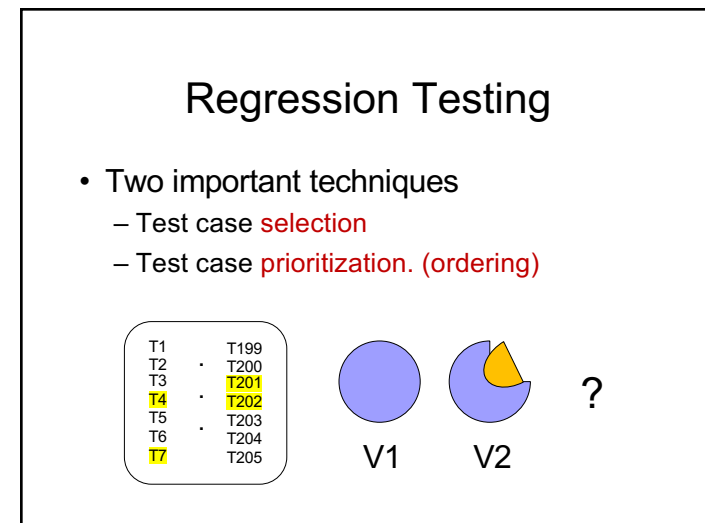
33



34



35



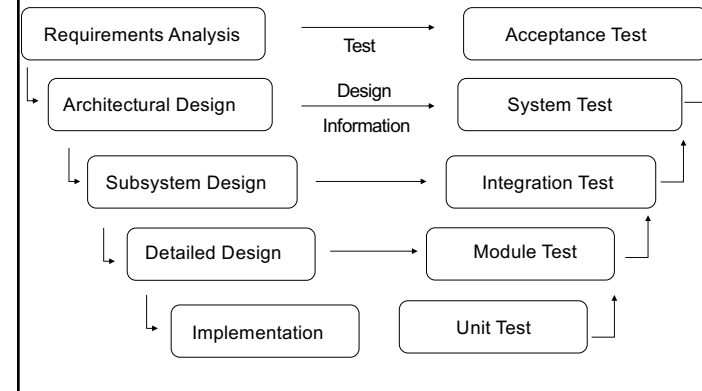
36

V-Model

- Relates testing and the different parts to the software development life cycle

37

V-Model



38

Early Test Design

- Mistakes in requirements and high-level design are costly
- Often found months or years after they are created
- May be dispersed throughout multiple versions/ and components making them hard to track down and expensive to fix

39

Concurrent Design/Test

- The expense of requirements-based faults means that it is important to do test planning concurrently with requirements analysis and design

40

Coverage Criteria

- Even
private static double (int A, int B, int C)
 - On a 32-bit machine, each variable has over 4 billion possible values
 - Over 80 octillion possible tests!!
 - Input space might as well be infinite
- Testers search a huge input space
 - Trying to find the fewest inputs that will find the most problems
- Coverage criteria give structured, practical ways to search the input space
 - Search the input space thoroughly
 - Not much overlap in the tests

Introduction to Software Testing, Edition 2 (Ch 2)

© Ammann & Offutt

41

41

Advantages of Coverage Criteria

- Maximize the “bang for the buck”
- Provide traceability from software artifacts to tests
 - Source, requirements, design models, ...
- Make regression testing easier
- Gives testers a “**stopping rule**” ... when testing is finished
- Can be well supported with powerful tools

Introduction to Software Testing, Edition 2 (Ch 2)

© Ammann & Offutt

42

42

Test Criterion

- **Collection of rules** along with a process that defines test requirements
 - Cover every statement
 - Cover every branch
 - Cover all widgets on the screen
 - Cover all edges in a graph
 - Cover all functional requirements

43

Test Requirements

- The things that **must be satisfied or covered** during testing
 - Each statement can be a test requirement
 - Each functional requirement can be a test requirement

44

Types of Structures for Defining Coverage

- Input domains
- Graphs
- Logic expressions
- Syntax descriptions

45

Specification and Structural Testing

- **Specification**– derive tests from the specification or external description (no need to have code). Often used at system level
- **Structural**– derive tests from code. Often used at unit level
- However, both can be used at either level and can be combined

46

Model Driven Test Design

- Four parts:
 - Test **design**
 - Test **automation**
 - Test **execution**
 - Test **evaluation**

47

Test **Design**

- **Criteria based**
 - Design tests to satisfy certain criteria
- **Human-based**
 - Design test values based on domain knowledge of the program and human knowledge of testing

48

Test Automation

- Using **executable scripts** for running test cases
- Automation can also be used for **test generation** and **design**
 - our book does not consider this “automation” – but we will

49

Test Execution

- Running tests and recording the results

50

Test Evaluation

- **Checking the results and reporting to developers**
- Who is responsible for determining and embedding the expected outputs?
- Test designers may not always know the expected outputs
- Test evaluators need to get involved early to help with this

51

Other Activities



Test management : Sets policy, organizes team, interfaces with development, chooses criteria, decides how much automation is needed, ...



Test maintenance : Save tests for reuse as software evolves

Requires cooperation of test designers and automators
Deciding when to trim the test suite is partly policy and partly technical – and in general, very hard !
Tests should be put in configuration control



Test documentation : All parties participate

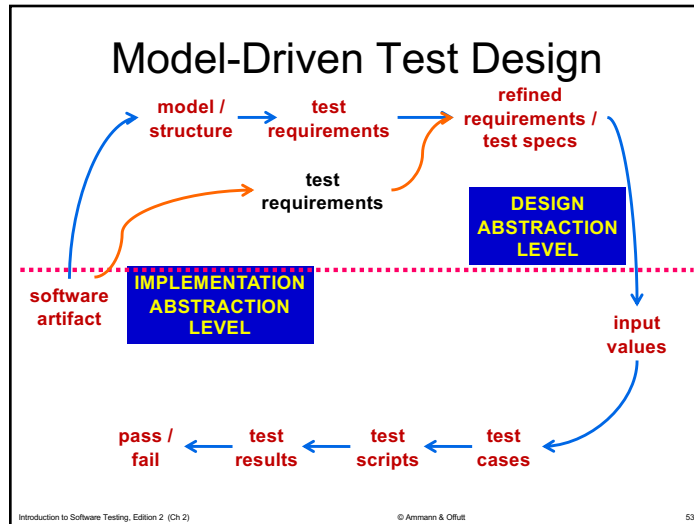
Each test must document “why” – criterion and test requirement satisfied or a rationale for human-designed tests
Ensure traceability throughout the process
Keep documentation in the automated tests

Introduction to Software Testing,
Edition 2 (Ch 2)

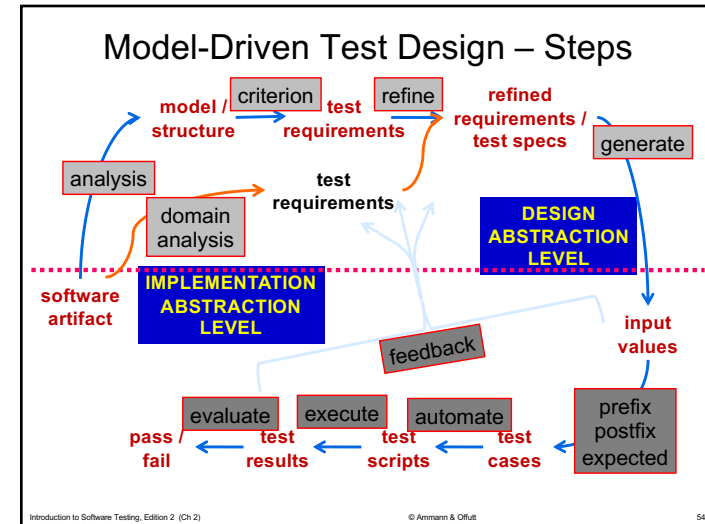
© Ammann & Offutt

52

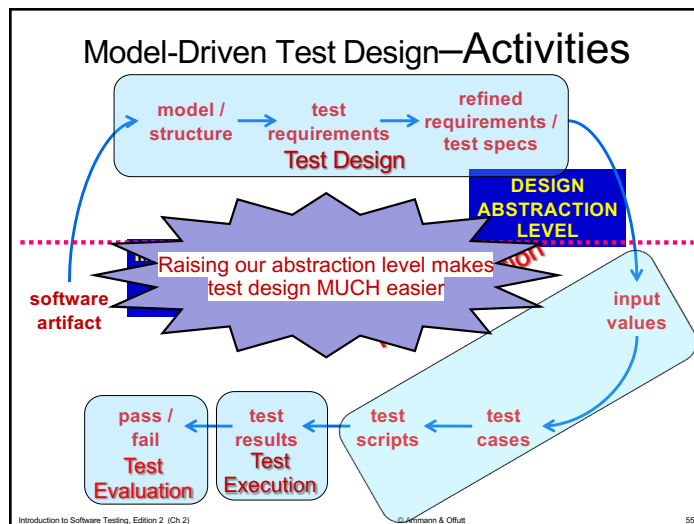
52



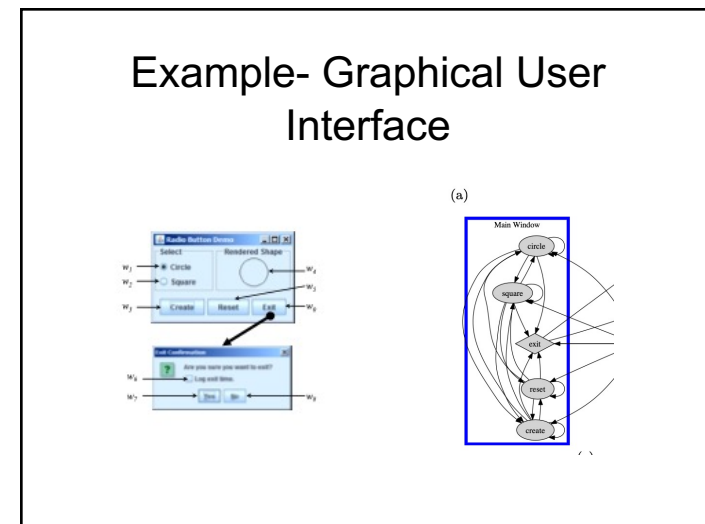
53



54



55



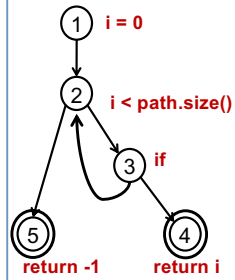
56

Small Illustrative Example

Software Artifact : Java Method

```
/**
 * Return index of node n at the
 * first position it appears,
 * -1 if it is not present
 */
public int indexOf (Node n)
{
    for (int i=0; i < path.size(); i++)
        if (path.get(i).equals(n))
            return i;
    return -1;
}
```

Control Flow Graph



Introduction to Software Testing, Edition 2 (Ch 2)

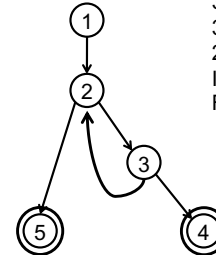
© Ammann & Offutt

57

57

Example (2)

Graph
Abstract version



Edges

1 2

2 3

3 2

3 4

2 5

Initial Node: 1

Final Nodes: 4, 5

6 requirements for
Edge-Pair Coverage

1. [1, 2, 3]

2. [1, 2, 5]

3. [2, 3, 4]

4. [2, 3, 2]

5. [3, 2, 3]

6. [3, 2, 5]

Test Paths

[1, 2, 5]

[1, 2, 3, 2, 5]

[1, 2, 3, 2, 3, 4]

Introduction to Software Testing, Edition 2 (Ch 2)

© Ammann & Offutt

58

58

Next Week

- Introduction to some testing tools
- Introduction to **Test Automation**

59